

Enterprise Google Cloud Data Platform Blueprint

VOLUME 04

Data Engineering Pipelines

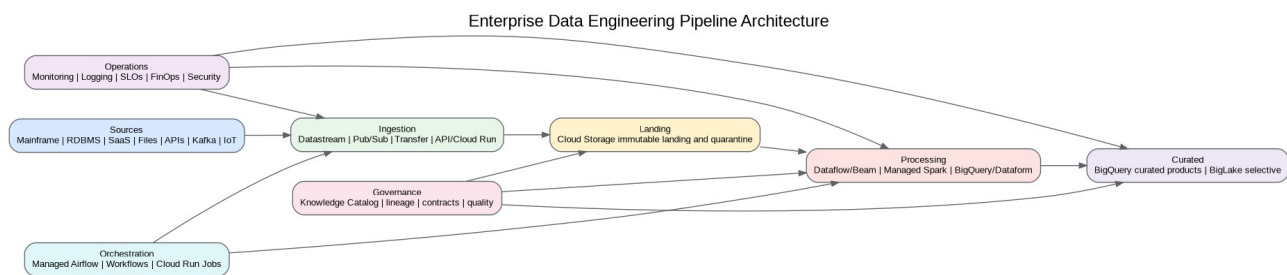


Figure 1. Enterprise data engineering pipeline architecture.

Architecture Baseline 1.0 | 12 July 2026
Confidential - Enterprise Reference Architecture

Purpose. Implementation-ready standards for batch, micro-batch, streaming, CDC, SQL ELT, Spark, orchestration, replay, quality, security, observability and support across the enterprise data platform.

Document Control

Field	Value
Document	Volume 04 - Data Engineering Pipelines
Series	Enterprise Google Cloud Data Platform Blueprint
Version	1.0
Status	Detailed reference architecture for adaptation and implementation
Date	12 July 2026
Owner	Enterprise Data Platform Architecture
Approvers	CDO, Enterprise Architecture, Data Governance, Security, Privacy, Platform Engineering, SRE and FinOps
Audience	Data engineers, platform engineers, architects, product teams, SRE, security, risk, audit and delivery partners
Review cycle	Quarterly and after material service, regulation, scale, SLO or architecture change
Primary dependencies	Volumes 01-03; Volume 05 data products; Volume 08 security; Volume 09 DevSecOps; Volume 10 FinOps; Volume 11 SRE; Volume 12 DR; Volume 13 Terraform
Scope	Pipeline patterns, services, engineering controls, implementation examples, operations and assurance
Out of scope	Detailed analytics semantics, AI pipelines, full landing-zone implementation and industry-specific designs
Guidance classification. Mandatory baseline means required unless an approved exception exists. Recommended pattern is the default for most workloads. Conditional pattern is used only when selection criteria are met. Future-state capability is included for planned evolution but is not required for initial production.	

Current product terminology

This volume uses Dataflow and Apache Beam for managed unified batch/stream processing; Managed Service for Apache Spark for serverless and cluster Spark patterns; Dataform for BigQuery SQL workflows; Managed Service for Apache Airflow for the service previously known as Cloud Composer; Pub/Sub for managed messaging; Datastream for supported CDC; and the BigQuery Storage Write API for direct high-scale writes. Implementation teams must validate current names, versions, regions, quotas, IAM roles, VPC Service Controls compatibility, CMEK support, SLAs and provider arguments before deployment.

Table of Contents

- 1. Executive Summary
- 2. Inherited Context, Scope and Assumptions
- 3. Data Engineering Strategy and Principles
- 4. Requirements and Success Measures
- 5. Target Pipeline Architecture
- 6. Pipeline Taxonomy and Selection
- 7. Batch and Micro-Batch Processing
- 8. Streaming, Event-Time, Windowing and State
- 9. Correctness: Deduplication, Idempotency and Late Data
- 10. Replay, Backfill, Reprocessing and Reconciliation
- 11. ETL and ELT Architecture
- 12. Dataflow and Apache Beam
- 13. Managed Service for Apache Spark and PySpark
- 14. Dataform and BigQuery SQL
- 15. Managed Airflow, Workflows and Cloud Run Jobs
- 16. Pub/Sub and Event Backbone
- 17. Datastream CDC
- 18. BigQuery Storage Write API
- 19. Orchestration, Dependencies and Metadata
- 20. Templates, Reusable Components and Connectors
- 21. Error Handling, Retry, DLQ, Quarantine and Checkpointing
- 22. Schema and Data Quality Validation
- 23. Observability, SLOs and Support
- 24. Performance and Cost Optimization
- 25. Security, Privacy and Compliance
- 26. Development, Testing and Deployment Standards
- 27. Service Assessments
- 28. Architecture Decision Records
- 29. Risks and Mitigations
- 30. Implementation Roadmap
- 31. Validation Criteria and Production Readiness
- 32. Cross-Volume Dependencies
- 33. Decisions and Next Actions

Appendices: Matrices, Code, Diagrams and References

1. Executive Summary

Volume 04 defines the engineering system that turns governed source data into reliable, testable and supportable data products. The architecture deliberately separates data-plane execution from control-plane orchestration, metadata, governance and operations. It standardizes a small set of managed services rather than allowing every domain to assemble an unrelated toolchain.

Mandatory design controls

- Use batch or micro-batch unless measured freshness requires streaming or CDC.
- Preserve immutable landing, source logs or retained events for recovery.
- Make contracts, event time, idempotency, quality, lineage, SLOs and cost visible in every pipeline.
- Promote immutable artifacts through environments; do not edit production pipelines in consoles.

Reference implementation standard

Area	Standard
Outcome	Enterprise default
Unified processing	Dataflow/Beam for new portable batch and streaming
Spark compatibility	Managed Service for Apache Spark when Spark-specific fit is demonstrated
Warehouse transformation	Dataform and BigQuery SQL
Orchestration	Managed Airflow for complex DAGs; Workflows for bounded API chains
Recovery	Replay, backfill and reconciliation are designed before production

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

2. Inherited Context, Scope and Assumptions

The enterprise processes 50-250 TB of new data per day, billions of events, thousands of pipelines and petabytes of retained data. AMER, EMEA and APAC data planes enforce residency and minimize cross-region movement. BigQuery is the primary analytical store; Cloud Storage provides landing, quarantine, replay and archive; Knowledge Catalog provides metadata; and domains own certified products under federated governance.

Mandatory design controls

- Group-based IAM and dedicated service accounts.
- Private-by-default connectivity, VPC Service Controls where supported and justified.
- CMEK for Restricted and Regulated classes or explicit regulatory requirements.
- Tier 0 RTO <1 hour/RPO <5 minutes; Tier 1 RTO <4 hours/RPO <15 minutes; Tier 2 RTO/RPO <24 hours.

Reference implementation standard

Area	Standard
Dimension	Inherited baseline
Regions	AMER, EMEA and APAC regional data planes
Environments	Development, test, staging, production and controlled sandbox
Classification	Public, Internal, Confidential, Restricted and Regulated
Ownership	Central platform plus federated domain product teams
Configuration authority	Terraform, Git and CI/CD

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

3. Data Engineering Strategy and Principles

The strategy is serverless-first, contract-first and product-oriented. The platform team owns paved roads, reusable connectors, runtime guardrails and common telemetry. Domain teams own pipeline logic, source understanding, quality rules, product SLOs and on-call participation. Complexity is introduced only when it creates measurable business value.

Mandatory design controls

- Bounded workloads are preferred for predictable recovery and cost.
- Streaming is a product commitment, not a technology preference.
- Every write is idempotent or reconciled.
- Lineage and run metadata are generated automatically.
- Security, privacy, SRE and FinOps controls are part of definition of done.

Reference implementation standard

Area	Standard
Principle	Concrete implication
Serverless-first	Use managed execution before persistent clusters
Contract-first	Reject or quarantine incompatible input before broad propagation
Replayable	Retain immutable source and deterministic code/configuration
Observable	Expose freshness, completeness, quality, latency, errors and cost
Domain accountable	Product owner accepts SLO and business correctness

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

4. Requirements and Success Measures

Pipeline requirements are expressed as measurable product outcomes rather than generic platform goals. Each production pipeline has a contract, owner, support tier, freshness and completeness SLO, quality gates, recovery source, capacity envelope, security classification and unit-cost measure.

Mandatory design controls

- At least 95% of new pipelines use approved templates.
- 100% of Tier 0/1 pipelines have tested replay and reconciliation.
- No production artifact is deployed outside CI/CD.
- Critical quality violations block certification and consumer publication.

Reference implementation standard

Area	Standard
Measure	Acceptance target
Template adoption	>=95% of eligible new pipelines
Contract coverage	100% production pipelines
Reconciliation	100% Tier 0/1 runs or continuous windows
Deployment evidence	Commit, tests, approvals and artifact digest
Owner routing	All alerts resolve to named product/on-call owner

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

5. Target Pipeline Architecture

The target architecture has four planes. The data plane moves and transforms data. The control plane schedules work and records dependencies. The governance plane holds contracts, classification, lineage and quality. The operations plane measures SLOs, incidents, capacity and cost. Separating these concerns prevents orchestration engines from becoming transformation engines and avoids embedding governance in unsearchable code.

Mandatory design controls

- Service projects are domain-aligned; shared networking, security, logging and build services remain centrally governed.
- Runtime identities are pipeline-specific or bounded to a product, not shared across unrelated domains.
- Inputs land before complex processing when recovery or legal evidence requires it.
- Curated publication is atomic and gated.

Reference implementation standard

Area	Standard
Plane	Components
Data	Pub/Sub, Datastream, Cloud Storage, Dataflow, Spark, BigQuery/Dataform
Control	Managed Airflow, Workflows, Cloud Run Jobs, run ledger
Governance	Knowledge Catalog, contracts, lineage, quality and stewardship
Operations	Monitoring, Logging, alerting, error budgets, FinOps and DR

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

6. Pipeline Taxonomy and Selection

A workload is classified by boundedness, freshness, change semantics, ordering, scale, recovery source and consumer contract. Batch remains the default for periodic bounded data. Micro-batch reduces latency while retaining bounded operation. Streaming is used for continuous events. CDC is used for database change replication. SQL ELT is used when data already resides in BigQuery.

Mandatory design controls

- Document the rejected alternatives and economics.
- Do not label a scheduled poll as CDC.
- Do not use streaming to compensate for unclear business requirements.
- Revisit pattern when sustained throughput, latency or operational evidence changes.

Reference implementation standard

Area	Standard
Pattern	Selection trigger
Batch	Freshness >=1 hour or bounded source delivery
Micro-batch	5-60 minute freshness with bounded partitions
Streaming	Seconds/minutes and event-driven consumers
CDC	Database log-based change replication
ELT	BigQuery-resident transformations and dimensional logic

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

7. Batch Processing

Batch pipelines ingest immutable manifests or bounded table snapshots, validate checksums and schemas, process into isolated staging, reconcile, then publish through partition replacement or idempotent MERGE. A run ID connects source manifest, code version, configuration, outputs, quality evidence and operational logs.

Mandatory design controls

- A rerun must not duplicate business records.
- Use explicit schemas and deterministic partition boundaries.
- Write to staging before mutating certified targets.
- Retain manifests and rejected-record evidence according to classification.

Reference implementation standard

Area	Standard
Stage	Mandatory action
Discover	Validate manifest, object list, checksum and expected arrival
Validate	Contract, malware where relevant, schema and basic quality
Process	Versioned Beam/Spark/SQL with run ID
Publish	Atomic partition replace or idempotent MERGE
Close	Reconciliation, lineage, certification and metrics

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

8. Micro-Batch Processing

Micro-batch is appropriate when near-real-time freshness is needed but continuous stateful processing is unnecessary. The pipeline processes event-time or ingestion-time intervals with a closed boundary, overlap margin and high-water mark. It preserves the operational advantages of bounded jobs while reducing delay.

Mandatory design controls

- Persist the high-water mark transactionally with successful publication.
- Include an overlap window and deduplicate by stable business/event key.
- Prevent concurrent runs for the same partition or define deterministic conflict handling.
- Alert on schedule drift and unprocessed intervals.

Reference implementation standard

Area	Standard
Design item	Standard
Interval	Driven by SLO and source behavior
Overlap	At least maximum observed source delay plus margin
State	Control table with last committed watermark
Write	Partition MERGE/replace
Recovery	Rerun scoped intervals using immutable source

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

9. Streaming Processing

Streaming pipelines consume continuous events through Pub/Sub, validate contract versions, assign event timestamps, apply watermarks/windows/state, deduplicate where required and write through Beam BigQueryIO or the Storage Write API. The design distinguishes processing semantics from source and sink semantics; exactly-once claims apply only to the full end-to-end path when verified.

Mandatory design controls

- Every event has immutable event_id, event_time, producer, schema_version and trace context.
- Use at-least-once when duplicates are harmless or cheaply removed; use exactly-once processing only where business value justifies it.
- Bound state with TTL and monitor watermark delay/backlog.
- Separate poison records from transient failures.

Reference implementation standard

Area	Standard
Concern	Decision
Semantics	At-least-once or exactly-once selected explicitly
Ordering	Ordering key only where business requirement exists
Backpressure	Autoscaling plus backlog/oldest-age alerts
Sink	Idempotent append/merge or offsets for committed streams
Recovery	Pub/Sub retention/snapshot or upstream event log

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

10. Event-Time Processing, Windowing and Watermarks

Event time is the business time when an event occurred. Processing time is when the platform observed it. Event-time windows prevent delayed network delivery from corrupting time-based measures. Watermarks estimate event-time completeness; allowed lateness controls how long state remains open for corrections.

Mandatory design controls

- Specify timestamp field, timezone, clock quality and fallback behavior in the contract.
- Choose fixed, sliding, session or global windows from business semantics.
- Late data is corrected, quarantined or published as an adjustment; it is never silently ignored for regulated measures.
- Measure watermark delay as a freshness SLI.

Reference implementation standard

Area	Standard
Window	Typical use
Fixed	Five-minute operational metrics and tumbling aggregates
Sliding	Rolling fraud/risk or activity measures
Session	User/device activity separated by inactivity gap
Global	Keyed state with explicit triggers; use cautiously
Late correction	Upsert/adjustment product with lineage

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

11. Stateful Processing and Checkpointing

Stateful processing tracks per-key context such as deduplication, sessions, joins or anomaly thresholds. State must be bounded, partitionable and recoverable. Dataflow-managed state and checkpointing are preferred over external mutable stores unless cross-pipeline sharing or specialized access requires one.

Mandatory design controls

- Define state key, TTL, growth model and deletion behavior.
- Avoid hot keys through key design or fan-out.
- Test worker restart, drain, update and regional recovery.
- Do not use unbounded global state.

Reference implementation standard

Area	Standard
Control	Implementation
State key	Stable, evenly distributed business key
TTL	Business lateness/replay window plus margin
Checkpoint	Managed by execution engine; validate recovery
Snapshot/update	Use supported Dataflow update/snapshot patterns
External state	Requires separate consistency, security and DR design

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

12. Deduplication and Idempotency

Deduplication removes repeated representations of the same event; idempotency ensures repeated processing has the same business effect. Both require stable keys and explicit retention. Infrastructure-level delivery guarantees do not protect external side effects, multi-sink transactions or producer duplicates.

Mandatory design controls

- Prefer producer-assigned immutable event IDs.
- Persist sink idempotency key before acknowledging external side effects.
- Define deduplication horizon from replay and business correction windows.
- Reconciliation remains mandatory for high-value products.

Reference implementation standard

Area	Standard
Pattern	Use
Beam state dedup	Bounded event horizon in streaming pipeline
BigQuery MERGE	Batch/micro-batch business-key upsert
Storage Write offsets	Committed streams needing exactly-once write
Outbox/inbox	Operational side effects and distributed integration
Partition replacement	Deterministic bounded rebuild

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

13. Late-Arriving Data

Late data is expected in global, mobile, IoT and partner ecosystems. Each product defines allowed lateness, correction semantics, consumer notification and whether regulatory reports are restated. The platform records both event time and processing/publication time so consumers can understand revisions.

Mandatory design controls

- Create a separate late-data metric and reason distribution.
- Do not mutate closed regulated periods without approval and audit evidence.
- Provide correction events or versioned partitions for downstream products.
- Test extreme lateness and clock-skew scenarios.

Reference implementation standard

Area	Standard
Class	Action
Within allowed lateness	Update active window/result
After window but before close	Publish correction/upsert
After financial/regulatory close	Exception workflow and restatement approval
Invalid timestamp	Quarantine with producer feedback

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

14. Replay, Backfill and Reprocessing

Replay reconsumes retained events; backfill processes historical data not previously loaded; reprocessing rebuilds data using corrected logic. All three use a unique replay ID, approved scope, immutable code/configuration, capacity plan, isolated staging and reconciliation before consumer publication.

Mandatory design controls

- Two-person approval for Tier 0/1 replay.
- Throttle or reserve capacity to protect live workloads.
- Prevent replay traffic from retriggering irreversible external actions.
- Retain before/after evidence and lineage.

Reference implementation standard

Area	Standard
Step	Evidence
Scope	Source range, partitions, keys and consumers
Prepare	Snapshot target and select artifact/config versions
Execute	Dedicated replay ID, labels and constrained capacity
Validate	Counts, sums, quality, duplicates and consumer checks
Publish/close	Atomic cutover, approval and retained report

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

15. Data Reconciliation

Reconciliation proves business completeness beyond technical job success. It compares source and target counts, control totals, key coverage, amounts, minimum/maximum timestamps and exception populations. Tolerances are approved by the data owner and steward, not chosen by the pipeline developer alone.

Mandatory design controls

- Tier 0/1 publication blocks on critical mismatches.
- Reconciliation logic is versioned independently from transformation logic.
- Store evidence in a restricted control dataset with retention.
- Aggregate checks do not replace record-level sampling or key coverage.

Reference implementation standard

Area	Standard
Check	Example
Volume	Source and target row/object/event counts
Value	Currency/quantity sums by business partition
Key	Missing, duplicate and orphan keys
Time	Source max change time versus target max applied time
Exception	Quarantine and unresolved correction totals

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

16. ETL and ELT Architecture

ETL is preferred when data must be transformed or protected before loading, when complex non-SQL processing is required, or when the target cannot safely receive raw data. ELT is preferred when governed data is already in BigQuery and transformations benefit from warehouse scale, SQL review and Dataform dependencies. Most enterprise products use both.

Mandatory design controls

- Apply tokenization/redaction before broad analytical landing when classification requires it.
- Use Dataform for SQL dependency graphs and assertions.
- Avoid moving BigQuery-resident data to Spark solely for familiar tooling.
- Avoid embedding complex business SQL in Airflow operators.

Reference implementation standard

Area	Standard
Need	Default
Streaming enrichment	Dataflow ETL
Complex file parsing	Dataflow or Managed Spark ETL
BigQuery dimensional model	Dataform ELT
Large joins already in BigQuery	BigQuery SQL
Specialized Spark library	Managed Spark conditional

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

17. Dataflow and Apache Beam

Dataflow is the primary managed engine for new unified batch and streaming pipelines. Apache Beam provides portable transforms, event-time semantics, windowing, state and testing. Flex Templates package code and dependencies as immutable container images while allowing controlled runtime parameters.

Mandatory design controls

- Use separate worker and launcher identities where supported.
- Use private IPs, approved subnets and restricted staging/temp buckets.
- Design for autoscaling, hot-key avoidance and connector semantics.
- Use drain/replacement for safe streaming updates and test snapshot compatibility where used.

Reference implementation standard

Area	Standard
Artifact	Required evidence
Beam library	Unit tests and versioned transform contracts
Flex Template	Image digest, metadata and parameter validation
Runtime	Region, network, service account, labels and options
Operations	Job health, backlog, watermark, errors and cost
Update	Compatibility test and rollback/drain plan

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

18. Managed Service for Apache Spark and PySpark

Managed Service for Apache Spark provides serverless batch and managed cluster deployment models. Serverless batches are the default for episodic jobs, migrations and bounded workloads. Persistent clusters require evidence such as long-running shared workloads, specialized initialization, locality or dependency constraints.

Mandatory design controls

- Pin runtime and library versions.
- Use private subnets and dedicated identities.
- Refactor legacy Spark jobs rather than copying inefficient cluster assumptions.
- Measure shuffle, skew, executor utilization and connector behavior with representative volume.

Reference implementation standard

Area	Standard
Deployment	Selection
Serverless batch	Episodic, isolated, autoscaled execution
Managed cluster	Persistent specialized workload with operational justification
Notebooks	Development only unless promoted as versioned jobs
Output	Prefer BigQuery or governed open-format storage
Recovery	Rerun immutable input into isolated target

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

19. Dataform and BigQuery SQL

Dataform is the default framework for versioned BigQuery SQL transformations. SQLX actions define tables, incremental logic, dependencies, documentation and assertions. Release and workflow configurations separate a tested code commit from environment-specific execution settings.

Mandatory design controls

- Set explicit default location to avoid accidental cross-region execution.
- Use assertions for uniqueness, non-null and custom business rules.
- Parameterize environment through compilation variables, not copied code.
- Design incremental predicates for late updates and backfills.

Reference implementation standard

Area	Standard
Action	Standard
Table/view	Document owner, grain and classification
Incremental	Unique key, boundary and full-refresh strategy
Assertion	Critical rules block downstream certification
Operation	Use only when table action is not appropriate
Release	Protected commit and environment workflow config

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

20. Managed Airflow and Apache Airflow

Managed Airflow orchestrates complex cross-service DAGs, dependencies, sensors, schedules and operational recovery. DAGs coordinate work; they do not contain large data payloads or perform transformation in scheduler memory. Environments are separated by risk and region and sized from measured scheduler/worker demand.

Mandatory design controls

- Set owner, retries, timeout, concurrency, pools and catchup intentionally.
- Avoid dynamic DAG explosion and top-level network calls.
- Use deferrable operators/sensors where supported.
- Deploy DAGs and dependencies through CI/CD with parse tests and version control.

Reference implementation standard

Area	Standard
Concern	Control
DAG design	Small tasks, clear boundaries and idempotent retries
Secrets	Secret Manager-backed connections; no DAG literals
Network	Private environment/access and approved egress
Observability	Task duration, failures, queue time and scheduler health
Recovery	Retry from safe boundary; do not falsify task state

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

21. Workflows and Cloud Run Jobs

Workflows is used for bounded API-centric orchestration, retries and compensation across managed services. Cloud Run Jobs executes bounded utilities such as manifest creation, metadata updates, schema conversion or reconciliation helpers. Neither replaces Dataflow or Spark for distributed data processing.

Mandatory design controls

- Retry only idempotent calls or add compensation.
- Use explicit timeouts and error branches.
- Keep utility images immutable and scanned.
- Pass references to data, not large payloads, through orchestration state.

Reference implementation standard

Area	Standard
Capability	Preferred tool
Complex recurring DAG	Managed Airflow
Short API chain	Workflows
Bounded container utility	Cloud Run Jobs
Distributed transform	Dataflow or Managed Spark
SQL graph	Dataform

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

22. Pub/Sub Event Backbone

Pub/Sub provides regionalized topics and subscriptions for event distribution, buffering and fan-out. Producers publish contract version and trace attributes. Consumers choose acknowledgment, retry, dead-letter, ordering and exactly-once subscription features based on business behavior. Retention and snapshots support bounded replay.

Mandatory design controls

- Do not rely on ordering unless ordering keys and downstream behavior are tested.
- Dead-letter topics require owner, retention, alert and replay workflow.
- Monitor backlog and oldest unacked age, not only message count.
- Design end-to-end idempotency even when exactly-once subscription delivery is enabled.

Reference implementation standard

Area	Standard
Element	Standard
Topic	Domain.event.version naming and classification
Subscription	Consumer-specific with retry/DLQ policy
Schema	Registered contract and compatibility policy
Attributes	event_id, event_time, schema_version, producer, trace
Replay	Snapshot/seek under controlled runbook

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

23. Datastream CDC

Datastream is the default for supported operational database CDC. It performs initial backfill and continuous change capture into BigQuery or Cloud Storage, depending on product pattern. Source preparation, private connectivity, log retention, schema evolution and reconciliation are shared responsibilities with database owners.

Mandatory design controls

- Validate source supplemental logging/replication settings and log retention.
- Treat backfill and ongoing CDC as separate capacity/risk phases.
- Monitor data staleness, stream/object errors and schema drift.
- Reserve or plan BigQuery CDC background processing where required.

Reference implementation standard

Area	Standard
Phase	Control
Prepare	Source logs, user privileges, network and capacity
Validate	Connection profiles and stream validation
Backfill	Object scope, throttling and reconciliation
CDC	Lag/staleness SLO and schema change handling
Recover	Restart/backfill affected objects and reconcile

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

24. BigQuery Storage Write API

The Storage Write API is the standard direct high-throughput BigQuery write interface. The default stream supports immediately available at-least-once writes. Application-created committed streams can use client-supplied offsets for exactly-once writes. Pending streams can support atomic batch commits.

Mandatory design controls

- Choose stream type and semantics explicitly.
- Persist and recover offsets for committed exactly-once streams.
- Handle row-level errors and poison records without retry storms.
- Respect schema update, stream count and throughput quotas.

Reference implementation standard

Area	Standard
Mode	Use
Default stream	At-least-once, low operational overhead
Committed stream + offsets	Exactly-once write where justified
Pending stream	Atomic batch commit
Beam BigQueryIO	Preferred integration from Dataflow
Validation	End-to-end duplicates and recovery tests

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

25. Orchestration, Dependency and Pipeline Metadata

Dependencies are represented as data availability and contract conditions, not arbitrary sleeps. A central run ledger records pipeline ID, product, run ID, interval, source version, code artifact, configuration, timestamps, status, row counts, quality outcome, lineage reference and cost attribution.

Mandatory design controls

- Use data intervals and event triggers rather than wall-clock assumptions.
- Do not create cyclic product dependencies.
- Publish technical lineage and business run evidence.
- Metadata writes must not expose sensitive payloads.

Reference implementation standard

Area	Standard
Metadata	Purpose
Pipeline registry	Owner, tier, contract, runtime and dependencies
Run ledger	Audit, recovery and operational truth
Quality results	Certification and exception workflow
Lineage	Source-to-output impact analysis
Cost labels	Product/domain/environment allocation

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

26. Templates, Reusable Components and Standard Connectors

Golden templates provide secure defaults for networking, identities, logging, labels, DLQ, run metadata and deployment. Reusable libraries implement contract parsing, standard error envelopes, metrics, tracing and BigQuery/Cloud Storage conventions. Teams can extend templates without forking the control surface.

Mandatory design controls

- Version templates semantically and publish migration notes.
- Keep business transformation code outside platform plumbing where practical.
- Test connector retry and schema behavior.
- Retire unsupported template versions through a managed lifecycle.

Reference implementation standard

Area	Standard
Asset	Ownership
Terraform module	Platform engineering
Beam/Spark library	Platform engineering with domain contributors
Pipeline template	Platform product team
Business transform	Domain product team
Connector certification	Platform, security and SRE

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

27. Error Handling, Retry, Dead Letter and Quarantine

Errors are classified as transient, poison data, contract violation, dependency failure, capacity failure or security event. Only transient failures are blindly retryable. Non-retryable records are stored in restricted quarantine with reason code, contract version, source reference, checksum and replay status.

Mandatory design controls

- Use bounded exponential backoff with jitter and deadlines.
- Preserve original record/object when policy permits; never log full sensitive payloads.
- Alert on DLQ age and growth.
- Quarantine access is more restrictive than normal landing access.

Reference implementation standard

Area	Standard
Class	Response
Transient	Retry with bounded policy
Poison record	Quarantine/DLQ and acknowledge
Contract break	Stop affected path and notify producer
Capacity/quota	Throttle, scale or approved quota change
Security anomaly	Stop, preserve evidence and invoke incident process

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

28. Checkpointing, Schema Validation and Data Quality

Checkpointing is handled by the execution engine where possible, while business checkpoints such as high-water marks and committed partitions are stored transactionally. Schema validation occurs at the boundary before broad propagation. Data-quality rules are layered from technical validity to business reconciliation and certification.

Mandatory design controls

- Schema compatibility policy is explicit: backward, forward or full.
- Critical quality rules fail publication, not merely alert.
- Reference datasets are versioned and included in lineage.
- Quality exceptions have owner, expiry and consumer disclosure.

Reference implementation standard

Area	Standard
Layer	Examples
Boundary	Parse, types, required fields and contract version
Standardized	Domain formats, reference values and duplicate keys
Curated	Grain, uniqueness, relationships and business calculations
Published	Freshness, completeness, reconciliation and certification
Operational	Quarantine age, exception backlog and rule runtime

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

29. Pipeline Observability and SLOs

Observability combines infrastructure, execution, data and product signals. Job success alone is insufficient. The platform measures input arrival, backlog, watermark, processing latency, output freshness, completeness, quality, reconciliation, failed records, resource saturation and unit cost.

Mandatory design controls

- Every alert maps to a runbook and named owner.
- Page only on user-impacting or rapidly worsening conditions.
- Use error budgets to govern release pace and reliability work.
- Correlate run ID and trace context across orchestration and processing.

Reference implementation standard

Area	Standard
SLI	Example
Availability	Successful publication windows / expected windows
Freshness	Publication time minus source event/change time
Completeness	Accepted expected records or control total ratio
Quality	Critical rule pass ratio
Recovery	Time from incident declaration to certified restoration

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

30. Performance and Cost Optimization

Performance tuning starts with data shape, partitioning, skew and connector semantics. Cost optimization preserves SLO and correctness; it does not disable necessary controls. Each pipeline reports units such as cost per GB processed, per million events, per completed partition or per certified product refresh.

Mandatory design controls

- Benchmark representative production distributions, not small uniform samples.
- Separate live streaming capacity from backfill capacity.
- Prune columns/partitions early and avoid repeated serialization.
- Control logging volume and metric cardinality.

Reference implementation standard

Area	Standard
Service	Primary tuning
Dataflow	Autoscaling, worker type, hot keys, shuffle, connector mode
Spark	Executor sizing, partitions, shuffle, skew and caching
BigQuery	Partition/cluster, incremental SQL, slots/reservations
Pub/Sub	Batching, flow control, retention and fan-out
Airflow	Scheduler/worker size, pools, deferrable waits

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

31. Security, Privacy and Compliance

Pipeline security applies Zero Trust to human, workload, network and data access. Each runtime uses a dedicated identity with only required source and target permissions. Sensitive data is minimized, tokenized or masked before broad exposure. Service perimeters and private access restrict exfiltration paths where supported.

Mandatory design controls

- No user-managed service account keys.
- Secrets are resolved from Secret Manager at runtime.
- CMEK is applied by classification and validated service support.
- Audit logs, code approvals and run evidence support regulated review.
- Privacy deletion and legal hold propagate through derived products and replay stores.

Reference implementation standard

Area	Standard
Boundary	Control
Identity	Federation, impersonation and dedicated runtime SA
Network	Private IP/API access, egress policy, VPC SC
Data	Classification, least privilege, RLS/CLS/masking downstream
Encryption	Google-managed default; CMEK/Cloud HSM where mandated
Evidence	Audit logs, lineage, approvals and retention

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

32. Development, Testing, Deployment and Support Standards

Engineering follows trunk-based or short-lived branch development, peer review, automated tests, immutable artifacts and staged promotion. Production support requires operational ownership, documentation, dashboards, alerts, runbooks, capacity limits, rollback and dependency contacts. Console changes are emergency exceptions and must be reconciled to code.

Mandatory design controls

- Unit-test pure transforms and parsing independently from services.
- Use isolated integration projects and masked/synthetic data.
- Test duplicates, late data, poison records, worker loss, replay and rollback.
- Promote the same artifact digest from staging to production.

Reference implementation standard

Area	Standard
Lifecycle	Gate
Develop	Lint, unit tests and contract fixtures
Build	Dependency, container, IaC and policy scans
Test	Integration, DQ, performance, resilience and security
Promote	Approvals and immutable artifact evidence
Operate	SLOs, support handoff and production review

Implementation teams must validate current service regions, quotas, lifecycle status, IAM roles, networking, CMEK and VPC Service Controls support before production. Any deviation from this baseline requires an owner, rationale, risk assessment, compensating controls and review date.

Figure 5-1 Architecture View

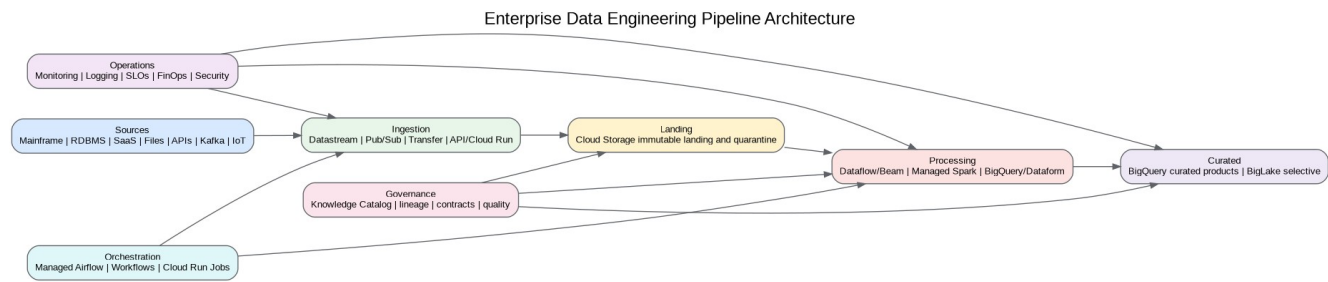


Figure 5-1. End-to-end data engineering architecture and cross-cutting planes.

Companion sources are provided in Mermaid, PlantUML, Graphviz DOT, SVG and editable draw.io formats. Connectors indicate logical flow; implementation must add region, project, service perimeter, environment and trust-boundary detail appropriate to the workload.

Figure 9-1 Architecture View

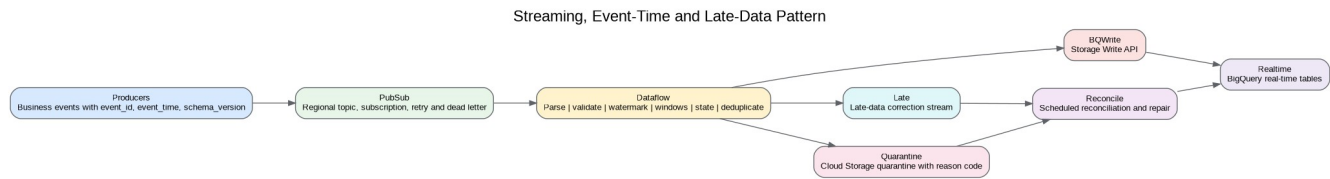


Figure 9-1. Streaming event-time, late-data and correction pattern.

Companion sources are provided in Mermaid, PlantUML, Graphviz DOT, SVG and editable draw.io formats. Connectors indicate logical flow; implementation must add region, project, service perimeter, environment and trust-boundary detail appropriate to the workload.

Figure 7-1 Architecture View



Figure 7-1. Batch and micro-batch validation, staging and publication pattern.

Companion sources are provided in Mermaid, PlantUML, Graphviz DOT, SVG and editable draw.io formats. Connectors indicate logical flow; implementation must add region, project, service perimeter, environment and trust-boundary detail appropriate to the workload.

Figure 23-1 Architecture View

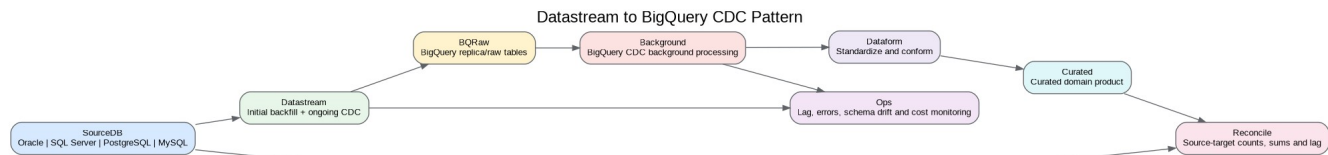


Figure 23-1. Datastream-to-BigQuery CDC and reconciliation pattern.

Companion sources are provided in Mermaid, PlantUML, Graphviz DOT, SVG and editable draw.io formats. Connectors indicate logical flow; implementation must add region, project, service perimeter, environment and trust-boundary detail appropriate to the workload.

Figure 20-1 Architecture View

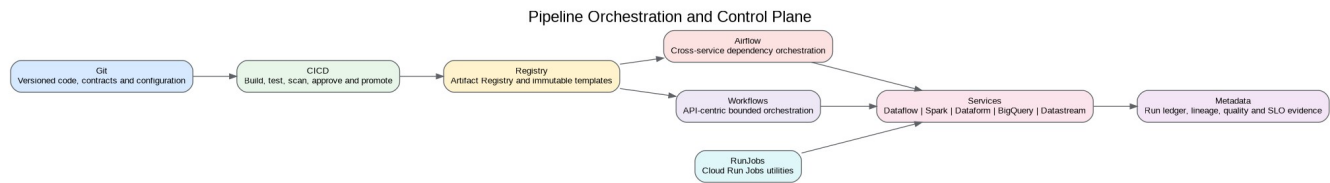


Figure 20-1. Orchestration, immutable artifacts and run metadata control plane.

Companion sources are provided in Mermaid, PlantUML, Graphviz DOT, SVG and editable draw.io formats. Connectors indicate logical flow; implementation must add region, project, service perimeter, environment and trust-boundary detail appropriate to the workload.

Figure 27-1 Architecture View



Figure 27-1. Classified error, repair, replay and reconciliation workflow.

Companion sources are provided in Mermaid, PlantUML, Graphviz DOT, SVG and editable draw.io formats. Connectors indicate logical flow; implementation must add region, project, service perimeter, environment and trust-boundary detail appropriate to the workload.

Figure 32-1 Architecture View

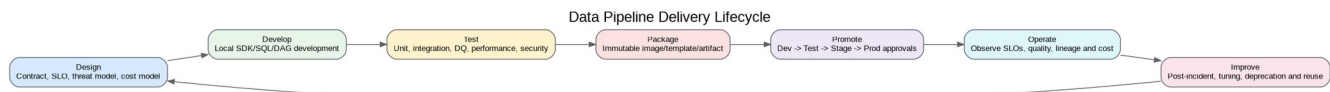


Figure 32-1. Pipeline engineering and production lifecycle.

Companion sources are provided in Mermaid, PlantUML, Graphviz DOT, SVG and editable draw.io formats. Connectors indicate logical flow; implementation must add region, project, service perimeter, environment and trust-boundary detail appropriate to the workload.

27. Service Assessments

The following structured assessments cover only services selected for the reference architecture. Product behavior changes over time; implementation teams must revalidate official documentation and record material findings in solution ADRs.

27.1 Dataflow

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Dataflow	Purpose	Managed execution for Apache Beam batch and streaming pipelines
Enterprise architecture role	Primary unified processing engine	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	Autoscaling workers, Streaming Engine, shuffle, templates, update/drain and managed connectors	Benefits	Serverless operations and unified event-time model
Limitations	Pipeline design, connector semantics, hot keys and dependency compatibility	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.2 Apache Beam

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Apache Beam	Purpose	Portable programming model for batch and streaming
Enterprise architecture role	Reusable engineering SDK and transform standard	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	PCollections, transforms, event time, windows, triggers, state, timers and testing	Benefits	Portability, common semantics and reusable libraries
Limitations	Runner-specific behavior, SDK/version compatibility and state growth	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.3 Managed Service for Apache Spark - Serverless

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Managed Service for Apache Spark - Serverless	Purpose	Managed serverless Spark batch execution
Enterprise architecture role	Conditional Spark engine for episodic/migration workloads	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	PySpark/Spark batches, autoscaled resources, private networking and connectors	Benefits	No cluster lifecycle and rapid isolation
Limitations	Runtime/library constraints, startup latency, quotas and migration inefficiency	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.4 Managed Service for Apache Spark - Clusters

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Managed Service for Apache Spark - Clusters	Purpose	Managed persistent Spark/Hadoop clusters
Enterprise architecture role	Conditional engine for persistent specialized workloads	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	Clusters, autoscaling, initialization and ecosystem compatibility	Benefits	Control for specialized runtimes and long-running jobs
Limitations	Operational overhead, idle cost and cluster hardening	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.5 Dataform

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Dataform	Purpose	Versioned BigQuery SQL workflow development
Enterprise architecture role	Default SQL ELT engineering layer	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	SQLX, dependencies, incremental tables, assertions, release/workflow configurations	Benefits	Governed SQL development and native BigQuery execution
Limitations	BigQuery-centric scope and careful incremental design	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.6 BigQuery

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	BigQuery	Purpose	Primary analytical storage and SQL execution
Enterprise architecture role	Curated data product store and ELT engine	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	Partitioning, clustering, MERGE, transactions, reservations, snapshots and audit	Benefits	Serverless scale, governance integration and high concurrency
Limitations	Location discipline, scan/slot cost and complex mutation patterns	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.7 Managed Service for Apache Airflow

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Managed Service for Apache Airflow	Purpose	Managed Apache Airflow orchestration
Enterprise architecture role	Complex dependency and schedule control plane	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	DAGs, operators, sensors, pools, retries, UI and monitoring	Benefits	Rich ecosystem and operational workflow visibility
Limitations	Environment cost, DAG quality, scheduler constraints and upgrades	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.8 Workflows

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Workflows	Purpose	Managed serverless API orchestration
Enterprise architecture role	Bounded control-flow and compensation	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	Connectors, expressions, retries, callbacks and execution history	Benefits	Low-operations orchestration for APIs
Limitations	Not a data-processing engine; execution/state limits	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.9 Cloud Run Jobs

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Cloud Run Jobs	Purpose	Managed execution for bounded containers
Enterprise architecture role	Control-plane utilities and small bounded processing	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	Versioned containers, tasks, environment variables, IAM and scheduling integration	Benefits	Simple immutable utilities without servers
Limitations	Not suited to large distributed/stateful processing	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.10 Pub/Sub

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Pub/Sub	Purpose	Managed messaging and event delivery
Enterprise architecture role	Enterprise event backbone and buffering layer	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	Topics, subscriptions, retention, ordering, retry, dead letter, schemas and replay	Benefits	Scalable decoupling and fan-out
Limitations	End-to-end idempotency, ordering constraints and retention bounds	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.11 Datastream

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	Datastream	Purpose	Managed change data capture
Enterprise architecture role	Supported operational database CDC and backfill	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	Connection profiles, streams, object selection, backfill and BigQuery/Cloud Storage destinations	Benefits	Managed log-based replication
Limitations	Source/version coverage, schema changes, connectivity and lag	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

27.12 BigQuery Storage Write API

Assessment field	Enterprise assessment	Assessment field	Enterprise assessment
Service name	BigQuery Storage Write API	Purpose	High-throughput BigQuery write interface
Enterprise architecture role	Direct streaming/batch write boundary	Business capabilities supported	Reliable ingestion, transformation, publication, recovery and governed data-product delivery.
Technical capabilities	Default, committed, buffered/pending streams, offsets and schemas	Benefits	Low-latency writes and optional exactly-once offsets
Limitations	Client offset management, quotas and poison-record handling	Service quotas	Validate project, region, API, worker/stream/job, throughput and control-plane quotas; request increases before representative scale tests.
Regional availability considerations	Deploy in the approved AMER, EMEA or APAC data plane; verify exact feature/runtime availability and avoid unapproved cross-region transfer.	High-availability characteristics	Use the managed service availability model and remove single points in orchestration, source access, staging, metadata and downstream dependencies.
Disaster-recovery considerations	Preserve immutable source, artifact, configuration, contract and run metadata; define rebuild/failover and test against the workload tier.	Security model	Dedicated runtime identity, least privilege, audit logging, private access and data controls by classification.
IAM requirements	Separate deployer, orchestrator and runtime roles; group-based human access; no broad primitive roles or user-managed keys.	Network requirements	Private IP/private Google access or approved private APIs; controlled egress, DNS and hybrid routes where required.
VPC Service Controls considerations	Validate current service support and ingress/egress rules; test CI/CD, orchestration and operational access across perimeters.	Encryption capabilities	Google-managed encryption by default; TLS in transit; protect staging, logs, metadata and error stores.
CMEK support	Apply where supported and required by Restricted/Regulated classification; validate region, service agent and rotation behavior.	Logging	Enable admin and required data-access audit logs plus structured application logs without sensitive payload leakage.
Monitoring	Track execution, latency, backlog/watermark, throughput, errors, resource saturation, quality, freshness and cost.	Alerting	Page on SLO/customer impact; ticket on trend/capacity; route to named product and platform owners with runbook links.
Cost drivers	Compute/runtime duration, data volume, storage/shuffle, API operations, retention, network movement and observability volume as applicable.	Cost-optimization techniques	Use the least-complex pattern, right-size/autoscale, prune early, reuse artifacts, control retention/logging and measure product-level unit cost.
Operational responsibilities	Platform owns paved road and service guardrails; domain owns logic, data correctness and product SLO; SRE owns operational framework.	Integration points	Cloud Storage, BigQuery, Pub/Sub, orchestration, Knowledge Catalog, Monitoring, Logging, IAM, KMS, CI/CD and Terraform as applicable.
Alternatives	Evaluate adjacent services or self-managed options only against correctness, SLO, security, recovery, skills, cost and operational evidence.	Selection criteria	Workload boundedness, freshness, event semantics, library compatibility, scale, region, recovery, security and total operational cost.
Best practices	Use immutable artifacts, explicit schemas/contracts, idempotent writes, bounded retry, quarantine, SLOs, tests and run metadata.	Reference architecture	Use the service in the target pipeline architecture and applicable pattern diagrams in this volume.
Sample configuration	See the companion examples, Terraform, SQL, YAML and README files in the implementation package.	Common pitfalls	Assuming managed means no design, mixing orchestration and processing, ignoring quotas/regions and declaring exactly-once without end-to-end proof.
Anti-patterns	Console-authored production, shared broad service accounts, unbounded retries/state, ownerless DLQs, destructive reruns and missing reconciliation.	Production-readiness checklist	Current docs validated; owner/tier/contract set; IAM/network/KMS approved; load, failure, replay and rollback tests pass; dashboards/runbooks active.

28. Architecture Decision Records

ADR	Decision title	Reference decision
ADR-0401	Select batch, micro-batch, CDC or streaming from measured freshness and recovery requirements	Use the least complex pattern that meets business freshness, ordering, volume and RPO; streaming is not the default for all data.
ADR-0402	Adopt a hybrid ETL and ELT strategy	Use Dataflow/Managed Spark for ingestion and complex processing; use Dataform/BigQuery SQL for warehouse-native transformations.
ADR-0403	Use Dataflow and Apache Beam as the primary unified batch and streaming engine	Standardize reusable Beam libraries and deploy immutable Dataflow Flex Templates.
ADR-0404	Use Managed Service for Apache Spark selectively with serverless-first execution	Choose serverless batches for episodic Spark; choose managed clusters only for persistent, specialized or migration constraints.
ADR-0405	Use Dataform as the default BigQuery SQL development and dependency framework	Version SQLX, assertions, release configurations and workflow invocations.
ADR-0406	Use Managed Airflow for complex DAGs and Workflows for bounded API orchestration	Avoid centralizing all workflow logic in one orchestrator and avoid processing data in orchestration engines.
ADR-0407	Use Pub/Sub as the managed event backbone with idempotent consumers	Require event IDs, schemas, bounded retry, DLQ, retention and explicit ordering decisions.
ADR-0408	Use Datastream for supported operational database CDC	Prefer managed backfill and continuous change capture over custom log readers.
ADR-0409	Use BigQuery Storage Write API for new high-scale direct streaming writes	Use committed streams and offsets only where end-to-end exactly-once value justifies operational complexity.
ADR-0410	Make event-time, watermarks, windows and allowed lateness part of the data contract	Define correction and replay behavior rather than silently dropping or overwriting late data.
ADR-0411	Deploy immutable pipeline artifacts and golden templates through CI/CD	Promote the same tested image/template/SQL/DAG commit across environments with policy evidence.
ADR-0412	Treat replay, backfill and reconciliation as first-class production capabilities	Every Tier 0/1 pipeline maintains immutable recovery sources, scoped replay controls and signed reconciliation evidence.

Complete ADRs using the mandated 26-field format are included as individual Markdown files in the companion package. They must be reviewed when a service capability, regulatory requirement, regional design, workload profile or exception materially changes.

29. Risks and Mitigations

ID	Risk	Likelihood	Impact	Mitigation	Owner
R4-01	Duplicate or missing records across retries	Medium	High	Event IDs, offsets, idempotent sinks, reconciliation	Platform + domain
R4-02	Late data changes published results	High	High	Event-time contract, allowed lateness, correction path	Domain
R4-03	Schema drift breaks consumers	Medium	High	Compatibility policy, contract CI, quarantine	Producer owner
R4-04	Airflow becomes centralized bottleneck	Medium	Medium	Domain DAG ownership, pools, bounded DAGs, Workflows for simple chains	Platform
R4-05	Uncontrolled backfill overloads production	Medium	High	Scoped partitions, reservations/quotas, staged replay and approvals	SRE + FinOps
R4-06	Spark migration preserves inefficient legacy jobs	High	Medium	Refactor assessment, serverless-first, performance tests	Engineering
R4-07	Sensitive payloads leak into logs/DLQ	Medium	High	Redaction, restricted quarantine, retention, DLP checks	Security
R4-08	Cost increases without product attribution	Medium	High	Labels, run ledger, unit economics, budgets	FinOps
R4-09	Regional service capability mismatch	Low	High	Predeployment region/quota validation and fallback pattern	Architecture
R4-10	Replay evidence incomplete	Medium	High	Immutable source, replay ID, approvals, reconciliation and lineage	SRE

Risk acceptance. A pipeline cannot enter production with an unowned High impact risk. Exceptions require a named risk owner, compensating controls, expiry date and evidence that consumers understand the residual risk.

30. Implementation Roadmap

Wave	Deliverables and exit criteria
0-30 days	Confirm inherited architecture, product terminology, regional availability, golden project/IAM/network patterns; define pipeline contract, SLO and run-ledger schemas.
31-90 days	Build Dataflow batch/stream templates, Dataform starter, Managed Spark batch, Pub/Sub/DLQ, Datastream and orchestration modules; establish CI/CD and integration test projects.
91-180 days	Migrate representative batch, streaming and CDC products; implement lineage, quality, reconciliation, dashboards, replay and cost allocation; run load and resilience tests.
181-270 days	Scale domain adoption, certify connectors, retire duplicated tools, optimize reservations/runtime patterns and complete Tier 0/1 recovery exercises.
271-365 days	Measure template adoption and unit economics, close control gaps, automate production-readiness evidence and review ADRs using observed workload data.

Migration waves

1. Wave A - Low-risk bounded batch products to prove templates and operating model.
2. Wave B - Dataform/BigQuery transformations and scheduled domain products.
3. Wave C - Supported database CDC with reconciliation and schema-change controls.
4. Wave D - Streaming products with event-time, replay and SLO evidence.
5. Wave E - Legacy Spark refactoring, platform consolidation and tool retirement.

31. Validation Criteria and Production Readiness

Control	Acceptance evidence	Mandatory
Owner and support model	Named product owner, engineer and on-call route	Yes
Contract and classification	Versioned contract, schema, classification and retention	Yes
Idempotency and replay	Duplicate test, immutable source, replay runbook	Yes
Quality and reconciliation	Critical assertions and source-target evidence	Yes
Security	Least privilege, private path, VPC SC/CMEK decision	Yes
Observability	SLIs, dashboards, alerts and runbook links	Yes
Performance/cost	Representative load test and unit-cost envelope	Yes
DR	Tier-aligned RTO/RPO test	Tier 0/1
Deployment/rollback	Immutable artifact, approvals and tested rollback	Yes
Documentation	Architecture, lineage, dependencies and known limits	Yes

Measurable acceptance tests

- Inject duplicate events and prove one business effect or documented reconciliation outcome.
- Inject late events inside and outside allowed lateness and prove correction behavior.
- Break schema compatibility and prove quarantine, alert and producer feedback.
- Terminate workers/tasks and prove checkpoint/retry behavior without silent loss.
- Run scoped replay/backfill while live processing continues within capacity/SLO envelope.
- Deploy a bad candidate and complete rollback to the prior immutable artifact.
- Reconcile source and target counts, values and keys for representative partitions.
- Prove least-privilege IAM by testing denied unauthorized paths and approved private access.

32. Cross-Volume Dependencies

Volume	Dependency from Volume 04	Contract back to Volume 04
01 Executive Architecture	Principles, NFRs, tiers and roadmap	Pipeline implementation evidence and refined decisions
02 Landing Zone	Projects, Shared VPC, IAM, KMS, VPC SC, logs and Terraform foundation	Runtime network/service-agent requirements
03 Enterprise Data Platform	Sources, layers, storage, BigQuery, governance and data products	Transformation, publication and recovery implementation
05 Data Mesh	Domain ownership, contracts, SLOs and product lifecycle	Reusable paved roads and product telemetry
06 Analytics	Curated/semantic freshness and certified measures	Consumer demand and query/performance feedback
07 AI/GenAI	Feature, RAG and training-data pipelines	AI data-quality and lineage requirements
08 Security	Zero Trust, classification, keys, perimeters and audit	Service-specific runtime/flow controls
09 DevSecOps	Build, scan, promote and supply-chain controls	Pipeline test/artifact requirements
10 FinOps	Allocation, budgets, reservations and unit economics	Pipeline cost drivers and labels
11 SRE	SLIs, alerts, on-call and incident management	Pipeline SLOs and runbooks
12 DR	Recovery tiers, failover and exercises	Replay/backfill/reconciliation mechanisms
13 Terraform	Golden modules, state and policy as code	Resource/module requirements
14 Industry	Industry controls and reference use cases	Reusable production pipeline patterns

33. Decisions and Next Actions

Approved reference decisions

- Dataflow/Apache Beam is the primary new unified batch and streaming engine.
- Managed Service for Apache Spark is conditional and serverless-first.
- Dataform/BigQuery SQL is the default warehouse-native transformation framework.
- Managed Airflow orchestrates complex DAGs; Workflows handles bounded API chains; Cloud Run Jobs handles bounded utilities.
- Pub/Sub is the event backbone; Datastream is the supported CDC default; Storage Write API is the high-scale BigQuery write boundary.
- Every Tier 0/1 pipeline is replayable, reconciled, observable, secure and deployed as an immutable artifact.

Immediate next actions

- Ratify pipeline contract, SLO and run-ledger schemas.
- Validate service regions, quotas, IAM, CMEK and VPC Service Controls for the first three regional patterns.
- Select one batch, one CDC and one streaming pilot data product.
- Build and certify the golden Dataflow, Dataform, Spark, Pub/Sub and orchestration modules.
- Execute duplicate, late-data, failure, replay, rollback, security and cost tests before production.
- Update the blueprint deliverable tracker to mark Volume 04 as the authoritative data engineering baseline.

Appendix A. Companion Implementation Package

The package includes executable starters and design artifacts. Each example README contains purpose, prerequisites, configuration, error handling, logging, security, testing, deployment, rollback and production limitations as required by the master prompt.

Artifact	Type
examples/airflow/README.md	.md
examples/airflow/pipeline_dag.py	.py
examples/beam_batch/README.md	.md
examples/beam_batch/batch_pipeline.py	.py
examples/beam_streaming/Dockerfile	text
examples/beam_streaming/README.md	.md
examples/beam_streaming/metadata.json	.json
examples/beam_streaming/pipeline.py	.py
examples/beam_streaming/requirements.txt	.txt
examples/bigquery/README.md	.md
examples/bigquery/reconciliation.sql	.sql
examples/bigquery/scd2_merge.sql	.sql
examples/cicd/README.md	.md
examples/cicd/cloudbuild.yaml	.yaml
examples/cloud_run_job/Dockerfile	text
examples/cloud_run_job/README.md	.md
examples/cloud_run_job/main.py	.py
examples/contracts/README.md	.md
examples/contracts/pipeline_contract.yaml	.yaml
examples/dataform/README.md	.md
examples/dataform/definitions/orders_curated.sqlx	.sqlx
examples/dataform/workflow_settings.yaml	.yaml
examples/datastream/README.md	.md
examples/datastream/main.tf	.tf

Additional artifacts omitted from the table for space: 13. See package manifest for the complete inventory.

Appendix B. Matrices and Registers

Matrix	Rows	Format
connector_standards.csv	5	CSV - editable and machine-readable
cost_driver_matrix.csv	6	CSV - editable and machine-readable
data_quality_rules.csv	6	CSV - editable and machine-readable
error_handling_matrix.csv	5	CSV - editable and machine-readable
governance_raci.csv	6	CSV - editable and machine-readable
iam_matrix.csv	6	CSV - editable and machine-readable
pipeline_slo_catalog.csv	3	CSV - editable and machine-readable
pipeline_taxonomy.csv	6	CSV - editable and machine-readable
production_readiness_checklist.csv	10	CSV - editable and machine-readable
promotion_matrix.csv	4	CSV - editable and machine-readable
reference_register.csv	11	CSV - editable and machine-readable
risk_register.csv	10	CSV - editable and machine-readable
service_selection_matrix.csv	6	CSV - editable and machine-readable
source_to_target_mapping.csv	5	CSV - editable and machine-readable
testing_matrix.csv	7	CSV - editable and machine-readable

Appendix C. Official Reference Register

Reference	Official location
Dataflow pipeline best practices	https://cloud.google.com/dataflow/docs/guides/pipeline-best-practices
Dataflow streaming modes	https://cloud.google.com/dataflow/docs/guides/streaming-modes
Dataflow Flex Templates	https://cloud.google.com/dataflow/docs/guides/templates/configuring-flex-templates
Managed Service for Apache Spark	https://cloud.google.com/dataproc-serverless/docs
Dataform SQL workflows	https://cloud.google.com/dataform/docs/sql-workflows
Dataform assertions	https://cloud.google.com/dataform/docs/assertions
Managed Service for Apache Airflow	https://cloud.google.com/composer/docs
Pub/Sub exactly-once delivery	https://cloud.google.com/pubsub/docs/exactly-once-delivery
Pub/Sub dead-letter topics	https://cloud.google.com/pubsub/docs/dead-letter-topics
Datastream BigQuery destination	https://cloud.google.com/datastream/docs/configure-bigquery-destination
BigQuery Storage Write API	https://cloud.google.com/bigquery/docs/write-api

Product names, previews, quotas, locations, SLAs, IAM permissions, Terraform arguments, pricing, CMEK and VPC Service Controls support can change. Implementation teams must validate current official documentation and record material differences in workload ADRs.

Appendix D. Document Completion Statement

Volume 04 is the authoritative data engineering pipeline baseline for the Enterprise Google Cloud Data Platform Blueprint. It establishes strategy, taxonomy, batch, micro-batch, streaming, event-time, state, correctness, recovery, ETL/ELT, selected service patterns, orchestration, metadata, reusable templates, connectors, errors, quality, observability, SLOs, performance, cost, security, development, testing, deployment, support, service assessments, ADRs, risks, roadmap, validation and readiness.

Completion condition. This reference architecture is ready for enterprise review and workload adaptation. It is not a substitute for discovery, data-contract approval, security/privacy assessment, regional and quota validation, performance testing, current-product validation or production readiness review.